

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

VIỆN TRÍ TUỆ NHÂN TẠO

BÁO CÁO MÔN HỌC

KỸ THUẬT VÀ CÔNG NGHỆ DỮ LIỆU LỚN

ĐỀ TÀI:

**HỆ THỐNG PHÁT HIỆN GIAN LẬN THẺ TÍN
DỤNG THEO THỜI GIAN THỰC TRONG
MOBILE BANKING SỬ DỤNG APACHE KAFKA**

Nhóm sinh viên thực hiện:

1. Doãn Nam Khánh MSV: 24022360 (Lớp: AI2-TC)

2. Đỗ Đức Hùng MSV: 24022342 (Lớp: AI2-TC)

Giảng viên hướng dẫn:

HÀ NỘI - 2026

MỤC LỤC

MỞ ĐẦU	3
1 TỔNG QUAN	4
1.1 Giới thiệu	4
1.1.1 Tại sao việc phát hiện gian lận thời gian thực lại quan trọng? . . .	4
1.1.2 Mục tiêu của dự án	4
1.1.3 Phạm vi dự án	5
1.2 Một số nghiên cứu liên quan	5
1.2.1 Sự chuyển dịch công nghệ và các kỹ thuật AI	5
1.2.2 Thách thức dữ liệu và nền tảng Big Data	6
2 PHƯƠNG PHÁP VÀ KIẾN TRÚC HỆ THỐNG	7
2.1 Các công nghệ sử dụng	7
2.1.1 Apache Kafka	7
2.1.2 Apache Zookeeper	8
2.1.3 Hadoop File System (HDFS)	8
2.1.4 Apache Spark	9
2.1.5 PostgreSQL	9
2.1.6 Streamlit	10
2.1.7 Docker	10
2.2 Thu thập và xử lý dữ liệu	11
2.2.1 Nguồn dữ liệu (Dataset)	11
2.2.2 Quy trình tiền xử lý và biến đổi dữ liệu	11
2.3 Mô hình hóa và dự đoán	12
2.3.1 Cơ chế hoạt động của XGBoost	12
2.3.2 Cấu hình và Tối ưu hóa siêu tham số (Hyperparameters)	13
2.3.3 Ưu điểm của XGBoost khi áp dụng vào hệ thống	13
2.4 Kiến trúc và Quy trình triển khai hệ thống	14
2.4.1 Kiến trúc hệ thống (System Architecture)	14
2.4.2 Quy trình thực hiện và vận hành luồng dữ liệu	15

2.5	Kết quả thực nghiệm và Giao diện giám sát	16
2.5.1	Kết quả huấn luyện và đánh giá mô hình phân loại	16
2.5.2	Thiết kế và Triển khai Giao diện Giám sát thời gian thực (Real-time Dashboard)	17
3	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	20
3.1	Kết luận	20
3.2	Hướng phát triển	21
	TÀI LIỆU THAM KHẢO	22
	PHÂN CÔNG NHIỆM VỤ	23

MỞ ĐẦU

Sự tăng trưởng nhanh chóng của các giao dịch trực tuyến và dịch vụ Mobile Banking đặt ra nhiều thách thức và rủi ro đáng kể về bảo mật, đặc biệt là vấn đề gian lận thẻ tín dụng. Trong môi trường giao dịch với tốc độ và khối lượng khổng lồ, các hệ thống tính truyền thống không còn đáp ứng được yêu cầu cảnh báo tức thời.

Dự án này xây dựng một pipeline có khả năng mở rộng và hoạt động theo thời gian thực (real-time) để mô phỏng, phát hiện và cảnh báo các giao dịch gian lận. Bằng cách tận dụng môi trường dữ liệu lớn với Apache Kafka để truyền phát thông điệp và Apache Spark (PySpark Structured Streaming) để xử lý luồng, hệ thống thực hiện phân tích dữ liệu ngay tại thời điểm giao dịch xảy ra. Chúng tôi tích hợp mô hình học máy (XGBoost) trực tiếp vào luồng dữ liệu để đánh giá xác suất gian lận, sau đó lưu trữ kết quả vào PostgreSQL và hiển thị cảnh báo thông qua bảng điều khiển Streamlit trực quan. Kết quả của dự án chứng minh tính hiệu quả của framework đề xuất, cung cấp khả năng giám sát dòng tiền và nhận diện rủi ro tấn công hệ thống với độ trễ tối thiểu.

CHƯƠNG 1: TỔNG QUAN

1.1. Giới thiệu

1.1.1. Tại sao việc phát hiện gian lận thời gian thực lại quan trọng?

Phát hiện gian lận thẻ tín dụng đóng vai trò sống còn đối với các tổ chức tài chính nhằm giảm thiểu tổn thất kinh tế và bảo vệ uy tín thương hiệu. Trong một môi trường giao dịch liên tục (24/7) như Mobile Banking, khả năng nhận diện và chặn đứng các giao dịch đáng ngờ trong vài giây có thể cứu vãn hàng triệu đô la. Những thách thức trong môi trường dữ liệu lớn đối với bài toán này bao gồm:

- **Tốc độ dữ liệu (Velocity):** Dữ liệu giao dịch đổ về liên tục, đòi hỏi hệ thống phải xử lý tức thời (real-time) để cảnh báo kịp thời trước khi tiền bị tẩu tán.
- **Mất cân bằng dữ liệu:** Tỷ lệ giao dịch gian lận thường cực kỳ thấp ($< 0.2\%$) so với giao dịch hợp lệ, gây khó khăn cho các mô hình phân loại truyền thống.
- **Chi phí tính toán:** Đưa mô hình Machine Learning vào môi trường xử lý luồng (streaming) dễ dẫn đến quá tải bộ nhớ và tắc nghẽn hệ thống nếu không được kiến trúc tốt.

1.1.2. Mục tiêu của dự án

- **Xây dựng hệ thống luồng dữ liệu:** Phát triển một pipeline dữ liệu đầu cuối sử dụng Apache Kafka làm Message Queue và Spark Streaming để xử lý dữ liệu.
- **Tích hợp Machine Learning:** Triển khai mô hình học máy Scikit-learn/XGBoost để đánh giá và phân loại xác suất gian lận trên từng giao dịch.
- **Giám sát trực quan:** Xây dựng một Dashboard cảnh báo theo thời gian thực, cung cấp góc nhìn toàn cảnh về dòng tiền và mức độ rủi ro của hệ thống.

1.1.3. Phạm vi dự án

Dự án bao gồm:

- Dự án tập trung vào việc **mô phỏng luồng dữ liệu** từ tập dữ liệu tĩnh (`creditcard.csv`) để tạo ra môi trường streaming tương tự Mobile Banking.
- Hệ thống xử lý trực tiếp các batch dữ liệu nhỏ (Micro-batching) và tính toán ra điểm rủi ro.
- Dự án chưa bao gồm việc triển khai chặn giao dịch trực tiếp trên Core Banking thực tế mà dừng ở mức độ cảnh báo và giám sát an ninh (Monitoring/Alerting).

Dự án không đi sâu vào:

- **Phát triển phần mềm thương mại hoàn chỉnh:** Hệ thống chỉ tập trung vào pipeline dữ liệu lớn và mô hình học máy mô phỏng. Dự án không xây dựng các chức năng phụ trợ như quản lý tài khoản người dùng, phân quyền bảo mật, hay giao diện cấu hình nâng cao.
- **Các nghiệp vụ xử lý sau gian lận:** Hệ thống chỉ dừng lại ở mức phát hiện và cảnh báo rủi ro. Các quy trình như tự động khóa thẻ, xác minh khách hàng hoặc xử lý khiếu nại không nằm trong phạm vi dự án.
- **Đa dạng hóa loại hình tội phạm tài chính:** Mô hình được huấn luyện chuyên biệt cho giao dịch thẻ tín dụng. Dự án không mở rộng sang các dạng gian lận khác như rửa tiền, chiếm đoạt tài khoản hay gian lận danh tính.
- **Phân tích dữ liệu hành vi ngoại vi:** Do giới hạn của bộ dữ liệu mô phỏng, mô hình không phân tích các yếu tố ngoài giao dịch tài chính như địa chỉ IP, định vị GPS hay thiết bị sử dụng (Device fingerprinting).

1.2. Một số nghiên cứu liên quan

1.2.1. Sự chuyển dịch công nghệ và các kỹ thuật AI

Nghiên cứu về phát hiện gian lận đang dịch chuyển mạnh mẽ từ kiểm duyệt lô (batch processing) truyền thống sang xử lý luồng (stream processing) theo thời gian thực. Các thuật toán Học máy (Machine Learning) được ứng dụng phổ biến bao gồm:

- **Cây quyết định & Rừng ngẫu nhiên:** Thể hiện sự vượt trội trong việc xử lý dữ liệu phi tuyến tính và chống quá khớp.
- **XGBoost (Extreme Gradient Boosting):** Tối ưu hóa hàm mất mát rất linh hoạt, cho tốc độ huấn luyện nhanh và hiệu suất cao với nhãn thiếu số (đây cũng là kỹ thuật cốt lõi của dự án).
- **Học sâu (Deep Learning):** Sử dụng Autoencoders hoặc LSTM để phát hiện bất thường dựa trên chuỗi hành vi mua sắm.

1.2.2. Thách thức dữ liệu và nền tảng Big Data

Đặc thù lớn nhất của bài toán này là tỷ lệ gian lận cực kỳ nhỏ ($< 0.2\%$). Để giải quyết, các nghiên cứu thường kết hợp:

- **Xử lý mất cân bằng & Đặc trưng:** Ẩn danh hóa dữ liệu bằng PCA, sau đó áp dụng các kỹ thuật sinh mẫu nhân tạo (SMOTE) hoặc phạt trọng số (Class Weights) để cân bằng dữ liệu huấn luyện.
- **Tích hợp Big Data:** Để đưa mô hình AI vào môi trường thực tế (production) mà không bị tắc nghẽn, việc sử dụng các nền tảng như **Apache Kafka** (để trung chuyển thông điệp thông lượng cao) và **Apache Spark Streaming** (tính toán phân tán độ trễ thấp tính bằng mili-giây) là điều kiện tiên quyết.

CHƯƠNG 2: PHƯƠNG PHÁP VÀ KIẾN TRÚC HỆ THỐNG

2.1. Các công nghệ sử dụng

Hệ thống được thiết kế dựa trên sự kết hợp của nhiều công nghệ mạnh mẽ trong hệ sinh thái Big Data và phát triển phần mềm hiện đại. Dưới đây là phân tích chi tiết về kiến trúc và vai trò của từng nền tảng được sử dụng:

2.1.1. Apache Kafka

Apache Kafka là một nền tảng xử lý luồng sự kiện phân tán (distributed event streaming platform) mã nguồn mở, được thiết kế để xử lý các luồng dữ liệu thời gian thực với độ trễ cực thấp và thông lượng cao.

a) Kiến trúc và các thành phần chính

- **Kafka Brokers:** Các máy chủ chịu trách nhiệm lưu trữ và phân phối dữ liệu. Mỗi broker có thể xử lý một phần dữ liệu và sao chép (replicate) với nhau để đảm bảo tính sẵn sàng cao.
- **Topics và Partitions:** Dữ liệu trong Kafka được phân loại vào các Topics (chủ đề). Để xử lý song song và tăng hiệu suất, mỗi Topic được chia nhỏ thành nhiều Partitions (phân vùng).
- **Producers:** Các ứng dụng hoặc luồng dịch vụ đẩy (publish) dữ liệu vào Kafka Topics.
- **Consumers:** Các ứng dụng đăng ký (subscribe) và đọc dữ liệu từ Topics.

b) Ứng dụng trong dự án

Trong dự án này, Kafka đóng vai trò là "trái tim" trung chuyển dữ liệu (Message Queue). Script mô phỏng đóng vai trò là **Producer**, liên tục đẩy các bản ghi giao dịch thẻ tín dụng vào topic `financial_transactions`. Hệ thống đảm bảo dữ liệu giao dịch được tiếp nhận mượt mà với độ trễ tính bằng mili-giây, không xảy ra tình trạng mất mát (data loss) ngay cả khi lưu lượng giao dịch tăng đột biến.

2.1.2. Apache Zookeeper

Zookeeper là một dịch vụ tập trung chuyên quản lý thông tin cấu hình, đặt tên và cung cấp cơ sở hạ tầng đồng bộ hóa phân tán cho các cụm máy chủ lớn.

a) Đặc điểm chính

- Giữ vai trò điều phối mạng lưới phân tán, theo dõi trạng thái "sống/chết" của các node trong cụm (cluster).
- Quản lý quá trình bầu chọn Leader (Leader Election) và duy trì tính nhất quán của metadata.

b) Ứng dụng trong dự án

Zookeeper được sử dụng làm dịch vụ nền tảng (backend service) cho Kafka. Nó theo dõi trạng thái của các Kafka Brokers, quản lý danh sách các Topics, Partitions và cấu hình hệ thống. Nhờ Zookeeper, cụm Kafka duy trì được độ ổn định và có khả năng tự động phục hồi khi một Broker gặp sự cố.

2.1.3. Hadoop File System (HDFS)

HDFS (Hadoop Distributed File System) là hệ thống tệp phân tán cốt lõi của hệ sinh thái Hadoop, được tối ưu hóa để lưu trữ và xử lý các tập dữ liệu khổng lồ trên cụm máy chủ thông thường.

a) Kiến trúc và đặc điểm

- **NameNode:** Nút quản lý trung tâm, lưu trữ metadata (cấu trúc thư mục, thông tin vị trí các khối dữ liệu).
- **DataNode:** Nơi lưu trữ thực tế các khối (blocks) dữ liệu. HDFS mặc định tạo nhiều bản sao (replicas) cho mỗi block để đảm bảo khả năng chịu lỗi (Fault Tolerance).
- **Thiết kế tối ưu:** Phù hợp với mô hình ghi một lần và đọc nhiều lần (Write-Once-Read-Many), phục vụ cực tốt cho các tác vụ phân tích batch.

b) Ứng dụng trong dự án

HDFS cung cấp nền tảng Data Lake đáng tin cậy. Trong hệ thống này, HDFS được dùng để lưu trữ an toàn tập dữ liệu lịch sử thẻ (creditcard.csv). Ngoài ra, nó đóng vai trò là

nơi lưu trữ trạng thái (Checkpoint location) cho Apache Spark Streaming, giúp luồng dữ liệu có thể khôi phục lại chính xác vị trí đang đọc (offset) nếu hệ thống phải khởi động lại.

2.1.4. Apache Spark

Apache Spark là một engine phân tích hợp nhất có tốc độ cực nhanh, chuyên xử lý dữ liệu quy mô lớn nhờ cơ chế tính toán trực tiếp trên bộ nhớ (In-memory computing).

a) Kiến trúc và thành phần chính

- **Spark Driver & Executors:** Quá trình thực thi được quản lý bởi Driver Program, phân phối các tác vụ tính toán (tasks) xuống các Worker Nodes chạy các Executors.
- **Spark Structured Streaming:** API mạnh mẽ cho phép xử lý dữ liệu luồng theo cơ chế mẻ nhỏ (Micro-batching), mang lại khả năng xử lý liên tục với độ trễ thấp và đảm bảo ngữ nghĩa "Exactly-once".

b) Ứng dụng trong dự án

Đóng vai trò là bộ não phân tích (Stream Processing Layer). PySpark được sử dụng để đăng ký nhận dữ liệu từ Kafka. Bằng cách giới hạn `maxOffsetsPerTrigger=3000`, hệ thống lấy dữ liệu theo từng micro-batch để tránh ngập lụt bộ nhớ. Đặc biệt, mô hình Machine Learning (XGBoost) được gọi trực tiếp thông qua cơ chế `foreachBatch` trên vùng nhớ của Spark Driver, cho phép dự đoán xác suất gian lận tức thời cho từng lô giao dịch.

2.1.5. PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở tiên tiến, nổi bật với tính ổn định cao và tuân thủ chặt chẽ tiêu chuẩn ACID.

a) Đặc điểm chính

- **Giao dịch ACID:** Đảm bảo tính toàn vẹn và độ tin cậy tuyệt đối của dữ liệu tài chính (Atomicity, Consistency, Isolation, Durability).
- **Hiệu năng cao:** Hỗ trợ các cơ chế lập chỉ mục (indexing) mạnh mẽ, giúp tăng tốc các truy vấn tổng hợp (aggregation) phức tạp.

b) Ứng dụng trong dự án

PostgreSQL đóng vai trò là kho dữ liệu phục vụ (Serving Layer). Sau khi PySpark chấm điểm gian lận, nó sẽ thực hiện chèn dữ liệu hàng loạt (batch insert) thông qua thư viện SQLAlchemy vào bảng `fraud_alerts`. CSDL này lưu trữ trạng thái thời gian thực của toàn bộ hệ thống và chịu tải cho các truy vấn liên tục từ Dashboard.

2.1.6. Streamlit

Streamlit là một micro-framework bằng Python, cho phép biến các kịch bản phân tích dữ liệu thành các ứng dụng web tương tác nhanh chóng.

a) Đặc điểm chính

- Cấu trúc mã nhẹ, không yêu cầu kiến thức phức tạp về Frontend (HTML/CSS/JS).
- Tính phản ứng (Reactivity) cao, tự động chạy lại luồng giao diện khi có thay đổi về trạng thái hoặc dữ liệu đầu vào.

b) Ứng dụng trong dự án

Được sử dụng để phát triển Real-time Dashboard. Ứng dụng thực hiện truy vấn (polling) liên tục vào PostgreSQL mỗi 3 giây để cập nhật các chỉ số KPIs. Bằng việc tích hợp các thư viện trực quan hóa như **Altair** (vẽ biểu đồ dòng tiền) và **Plotly** (vẽ đồng hồ đo rủi ro Gauge Chart), Streamlit mang lại một giao diện giám sát an ninh trực quan, hỗ trợ đắc lực cho nghiệp vụ ngân hàng.

2.1.7. Docker

Docker là nền tảng phần mềm cho phép đóng gói ứng dụng và các thư viện phụ thuộc vào trong các container độc lập, chuẩn hóa.

a) Đặc điểm chính

- **Cô lập môi trường:** Các container hoạt động độc lập, không xung đột tài nguyên với nhau hay với hệ điều hành máy chủ.
- **Nhẹ và di động:** Khởi động nhanh hơn máy ảo (VM) truyền thống và có thể chạy đồng nhất trên mọi môi trường (Local, Testing, hay Cloud).

b) Ứng dụng trong dự án

Docker và `docker-compose` được sử dụng để đóng gói toàn bộ các service kiến trúc của hệ thống (bao gồm Zookeeper, Kafka, PostgreSQL, Spark, và Streamlit Dashboard). Việc này giải quyết triệt để vấn đề "chạy được trên máy tôi nhưng lỗi trên máy bạn", cho phép triển khai, khởi động và dọn dẹp toàn bộ dự án Big Data phức tạp chỉ với một vài câu lệnh đơn giản.

2.2. Thu thập và xử lý dữ liệu

2.2.1. Nguồn dữ liệu (Dataset)

Dữ liệu được sử dụng trong dự án là tập **Credit Card Fraud Detection Dataset**, bao gồm các giao dịch thẻ tín dụng của chủ thẻ tại châu Âu được thực hiện trong vòng hai ngày vào tháng 9 năm 2013. Đây là một tập dữ liệu tiêu chuẩn trong các bài toán an ninh tài chính với các đặc điểm nổi bật sau:

- **Quy mô:** Tổng cộng 284,807 giao dịch.
- **Độ lệch dữ liệu cực lớn (Highly Imbalanced):** Giao dịch hợp lệ (Class 0) chiếm tỷ lệ áp đảo tới 99.827% (284,315 giao dịch), trong khi giao dịch gian lận (Class 1) chỉ chiếm một phần rất nhỏ là 0.172% (492 giao dịch).
- **Các đặc trưng (Features):**
 - **Time:** Số giây trôi qua tính từ thời điểm diễn ra giao dịch đầu tiên trong tập dữ liệu.
 - **Amount:** Giá trị tiền tệ thực tế của giao dịch.
 - **V1 đến V28:** Các đặc trưng ẩn đã được biến đổi bằng phương pháp Phân tích thành phần chính (PCA) nhằm bảo vệ quyền riêng tư và thông tin định danh cá nhân của khách hàng.
 - **Class:** Biến mục tiêu cần phân loại (0 là Hợp lệ, 1 là Gian lận).

2.2.2. Quy trình tiền xử lý và biến đổi dữ liệu

Hệ thống thực hiện xử lý dữ liệu theo một quy trình khép kín, từ việc mô phỏng luồng (streaming) đến chuẩn hóa phục vụ cho thuật toán học máy:

a) Mô phỏng luồng dữ liệu (Data Ingestion)

Sử dụng tập dữ liệu tĩnh `creditcard.csv` để giả lập môi trường giao dịch thay cho việc đầu nối trực tiếp với core Banking. Thành phần `transaction_producer.py` (đóng vai trò Kafka Producer) sẽ đọc tuần tự dữ liệu, định dạng lại thành các gói tin JSON và đẩy liên tục vào Kafka topic `financial_transactions` với nhịp độ mô phỏng thời gian thực.

b) Tiền xử lý và chuẩn hóa (Preprocessing)

Trong khi các đặc trưng từ V1 đến V28 đã được biến đổi qua PCA, hai đặc trưng `Time` và `Amount` lại có khoảng giá trị chênh lệch rất lớn. Nếu giữ nguyên, mô hình học máy dễ bị thiên lệch (bias) bởi các giá trị có biên độ lớn này. Để khắc phục, hệ thống sử dụng `StandardScaler` của thư viện `scikit-learn` để chuẩn hóa `Time` và `Amount` về phân phối chuẩn (trung bình $\mu = 0$, phương sai $\sigma^2 = 1$). Bộ tham số chuẩn hóa này được huấn luyện trên tập Train và xuất ra tệp `scaler.pkl` để đảm bảo tính nhất quán khi áp dụng trên luồng dữ liệu streaming.

c) Cấu trúc dữ liệu luồng (Micro-batching)

Dữ liệu thô từ Kafka được PySpark Structured Streaming kéo về theo từng mẻ nhỏ (micro-batch) với giới hạn `maxOffsetsPerTrigger=3000`. Trong quá trình xử lý mỗi mẻ (thông qua hàm `foreachBatch`), dữ liệu được chuyển đổi linh hoạt sang cấu trúc `Pandas DataFrame`. Tại không gian bộ nhớ của Spark Driver, bộ tiền xử lý `scaler.pkl` sẽ được tải lên và biến đổi dữ liệu ngay tức thì trước khi đẩy vào mô hình `XGBoost` để đưa ra phán quyết cuối cùng.

2.3. Mô hình hóa và dự đoán

Để giải quyết bài toán phân loại nhị phân trên tập dữ liệu mất cân bằng nghiêm trọng (giao dịch gian lận chỉ chiếm 0.172%), dự án đã triển khai thuật toán học máy **XGBoost** (**Extreme Gradient Boosting**). Đây là một kỹ thuật học tập hợp (ensemble learning) dựa trên cây quyết định (decision trees), nổi bật với tốc độ huấn luyện nhanh và khả năng kiểm soát hiện tượng quá khớp (overfitting) xuất sắc.

2.3.1. Cơ chế hoạt động của XGBoost

XGBoost hoạt động theo nguyên lý *Gradient Boosting*. Thuật toán sẽ xây dựng các cây quyết định một cách tuần tự, trong đó mỗi cây được tạo ra ở bước sau sẽ tập trung học

và sửa chữa những sai số (residual errors) mà các cây trước đó dự đoán sai. Bằng cách tối ưu hóa hàm mất mát (loss function) thông qua thuật toán giảm gradient (gradient descent), hệ thống liên tục nâng cao độ chính xác sau từng vòng lặp cho đến khi đạt được mô hình tối ưu.

2.3.2. Cấu hình và Tối ưu hóa siêu tham số (Hyperparameters)

Thách thức lớn nhất trong dự án là ngăn ngừa tình trạng mô hình thiên lệch hoàn toàn về lớp giao dịch hợp lệ do số lượng mẫu quá áp đảo. Để khắc phục, nhóm đã áp dụng cơ chế bù đắp trọng số cho lớp thiểu số thông qua tham số `scale_pos_weight`. Trọng số này được tính toán tự động dựa trên tỷ lệ giữa hai lớp trong tập huấn luyện:

$$\text{scale_pos_weight} = \frac{N_{\text{legitimate}}}{N_{\text{fraud}}} \approx 577.29$$

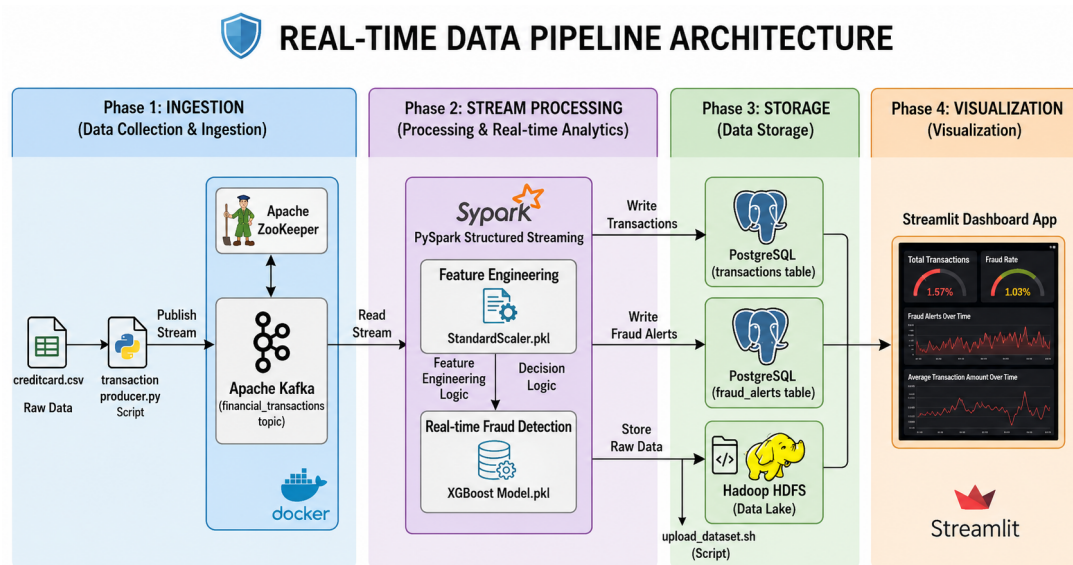
Các siêu tham số quan trọng khác được thiết lập để tối ưu hóa mô hình bao gồm:

- `n_estimators = 200`: Xây dựng tối đa 200 cây quyết định.
- `max_depth = 5`: Giới hạn chiều sâu của mỗi cây để tránh mô hình học vẹt (overfitting).
- `learning_rate = 0.1`: Tốc độ học (shrinkage factor) giúp mô hình hội tụ ổn định.
- `tree_method = 'hist'`: Sử dụng phương pháp phân loại dựa trên biểu đồ (histogram-based) giúp tăng tốc độ huấn luyện trên luồng dữ liệu lớn.
- `eval_metric = 'auc'`: Sử dụng diện tích dưới đường cong ROC làm tiêu chí đánh giá và tối ưu.

2.3.3. Ưu điểm của XGBoost khi áp dụng vào hệ thống

- **Tốc độ và hiệu năng cao**: Thuật toán hỗ trợ tính toán song song, vô cùng phù hợp khi tích hợp với PySpark để dự đoán độ trễ thấp trên luồng dữ liệu thời gian thực.
- **Xử lý mất cân bằng hiệu quả**: Khả năng tinh chỉnh linh hoạt hàm mất mát và bù đắp trọng số giúp mô hình đặc biệt nhạy bén với các giao dịch gian lận hiếm gặp, tránh việc dự đoán toàn bộ là lớp hợp lệ.

2.4. Kiến trúc và Quy trình triển khai hệ thống



Hình 2.1: Sơ đồ tổng quan kiến trúc hệ thống

2.4.1. Kiến trúc hệ thống (System Architecture)

Hệ thống được thiết kế theo mô hình kiến trúc xử lý dữ liệu lớn khép kín, phân tách rõ ràng thành 4 phân lớp chức năng nhằm đảm bảo tính toàn vẹn, khả năng mở rộng và tối ưu hóa hiệu năng:

- **Ingestion Layer (Tầng thu thập dữ liệu)**: Sử dụng tệp `creditcard.csv` để trích xuất dữ liệu, cấu phần `transaction_producer.py` giả lập các đợt phát dữ liệu (burst) liên tục dưới định dạng gói tin JSON. Các gói tin này được đẩy trực tiếp vào hệ thống hàng đợi thông điệp phân tán Apache Kafka Topic (`financial_transactions`) dưới sự điều phối và quản lý trạng thái chặt chẽ của Apache ZooKeeper.
- **Stream Processing Layer (Tầng xử lý luồng)**: Sử dụng PySpark Structured Streaming làm kiến trúc cốt lõi để lắng nghe dữ liệu từ Kafka theo cơ chế Micro-batching. Tại đây, luồng dữ liệu được chuẩn hóa thông qua bộ tiền xử lý `scaler.pkl` và chuyển đổi sang cấu trúc Pandas DataFrame trên vùng nhớ của Spark Driver để chạy mô hình phân loại XGBoost chấm điểm rủi ro trực tiếp.

- **Storage Layer (Tầng lưu trữ):** Toàn bộ lịch sử giao dịch kèm theo các nhãn xác suất gian lận (`fraud_probability`) được ghi hàng loạt (batch insert) vào hai bảng hệ thống `transactions` và `fraud_alerts` trong cơ sở dữ liệu PostgreSQL. Song song với đó, tập dữ liệu gốc được đồng bộ và tải lên hệ thống tệp tin phân tán Hadoop HDFS thông qua các script tự động để lưu trữ lâu dài và dự phòng rủi ro phần cứng.
- **Visualization Layer (Tầng trực quan hóa):** Ứng dụng Dashboard phát triển trên nền tảng Streamlit liên tục kết nối với PostgreSQL để truy xuất dữ liệu, cập nhật các chỉ số báo cáo và đưa ra cảnh báo thời gian thực tới người vận hành an ninh hệ thống.

2.4.2. Quy trình thực hiện và vận hành luồng dữ liệu

Quy trình triển khai và vận hành hệ thống được chia thành 4 giai đoạn chính yếu:

a) Lập kế hoạch và phân tích

- **Mục tiêu:** Mô phỏng luồng giao dịch thời gian thực, ứng dụng XGBoost để phát hiện gian lận và trực quan hóa cảnh báo qua Dashboard.
- **Yêu cầu:** Hệ thống xử lý độ trễ thấp, chịu tải cao (không gây tràn bộ nhớ) và có khả năng mở rộng linh hoạt.

b) Xây dựng các lớp kiến trúc

- **Ingestion Layer:** Khởi tạo Kafka Broker, tạo topic `financial_transactions` và chạy `transaction_producer.py` để bơm dữ liệu liên tục.
- **Stream Layer:** Chạy `fraud_detector.py` (PySpark) theo cơ chế Micro-batching, tích hợp mô hình XGBoost và `scaler.pkl` để chấm điểm rủi ro.
- **Storage Layer:** Cấu hình PostgreSQL (tạo sẵn bảng `transactions`, `fraud_alerts`) để lưu kết quả từ Spark và tải dữ liệu thô lên HDFS để dự phòng.
- **Serving Layer:** Lập trình `dashboard.py` bằng Streamlit, thiết lập truy vấn cơ sở dữ liệu 3 giây/lần để cập nhật biểu đồ trực quan.

c) Triển khai hạ tầng bằng Docker

- Sử dụng `Docker Compose` để đóng gói, cô lập và khởi chạy đồng loạt các dịch vụ lõi bao gồm: Apache ZooKeeper, Kafka Broker và PostgreSQL.

d) Kiểm thử và tối ưu hóa

- **Kiểm thử luồng khép kín:** Xác minh dữ liệu chạy thông suốt qua Data Pipeline: File CSV $\rightarrow$ Kafka $\rightarrow$ PySpark $\rightarrow$ PostgreSQL $\rightarrow$ Streamlit.
- **Tối ưu hóa:** Khắc phục lỗi `PyArrow TimeoutError` bằng lệnh `toPandas()`, triệt tiêu sai số dấu phẩy động của SQL và đồng bộ mốc thời gian giả lập với thời gian thực.

2.5. Kết quả thực nghiệm và Giao diện giám sát

2.5.1. Kết quả huấn luyện và đánh giá mô hình phân loại

Để chứng minh năng lực phân biệt và phát hiện tội phạm công nghệ cao, mô hình XGBoost Classifier đã được đánh giá một cách khách quan trên tập dữ liệu kiểm thử độc lập chiếm 20% tổng dung lượng mẫu. Kết quả kiểm định được mô tả chi tiết thông qua Ma trận nhầm lẫn (Confusion Matrix) dưới đây:

Bảng 2.1: Ma trận nhầm lẫn (Confusion Matrix) của mô hình XGBoost

Thực tế \ Dự đoán	Dự đoán Hợp lệ (0)	Dự đoán Gian lận (1)
Thực tế Hợp lệ (0)	56,848 (True Negative)	16 (False Positive)
Thực tế Gian lận (1)	16 (False Negative)	82 (True Positive)

Dựa trên ma trận thực nghiệm, hệ thống đạt được các chỉ số hiệu năng vô cùng ấn tượng:

- **Độ chính xác tổng thể (Accuracy):** Đạt mức tuyệt đối 99.94% trên toàn bộ tập dữ liệu, minh chứng cho khả năng vận hành chuẩn xác của thuật toán.
- **Độ thu hồi (Recall - Lớp gian lận):** Đạt 0.84, nghĩa là hệ thống bắt bài và phát hiện thành công 84% tổng số vụ gian lận thực tế xảy ra trong tập dữ liệu kiểm thử.
- **Độ chính xác cục bộ (Precision - Lớp gian lận):** Đạt 0.84, khẳng định rằng khi hệ thống phát ra một tín hiệu cảnh báo lừa đảo, có tới 84% khả năng đó chắc

chấn là một giao dịch gian lận thực sự, hạn chế tối đa việc làm phiền khách hàng hợp lệ.

- **Điểm số ROC-AUC:** Đạt mức 0.9778 (tiệm cận mức lý tưởng 1.0). Chỉ số AUC cực cao chứng minh năng lực phân tách xuất sắc giữa hai phân lớp dữ liệu của mô hình XGBoost, ngay cả khi đối mặt với sự chênh lệch mẫu nghiêm trọng. Hệ thống chỉ ghi nhận vỏn vẹn 16 trường hợp báo động sai (False Alarm) trên tổng số hơn 56,864 giao dịch thông thường.

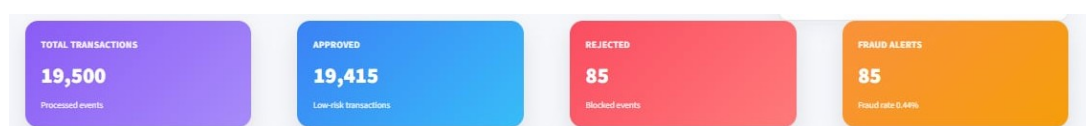
2.5.2. Thiết kế và Triển khai Giao diện Giám sát thời gian thực (Real-time Dashboard)

Bảng điều khiển được thiết kế với cơ chế tự động làm mới (Auto-refresh) và chia thành các phân hệ chức năng chính tương ứng với hệ thống đường ống dữ liệu:

Thẻ chỉ số trọng yếu (Key Performance Indicators - KPIs)

Hệ thống cung cấp 4 thẻ chỉ số được cập nhật liên tục (Live), phản ánh ngay lập tức khối lượng công việc và trạng thái vận hành mà hệ thống Streaming đang xử lý:

- **Tổng số giao dịch (Total Transactions):** Thống kê tổng lượng sự kiện đã xử lý qua hệ thống.
- **Hợp lệ (Approved):** Số lượng giao dịch được mô hình XGBoost phân loại là rủi ro thấp và được phê duyệt hệ thống.
- **Từ chối (Rejected) & Cảnh báo (Fraud Alerts):** Hiển thị số lượng giao dịch bị hệ thống phát hiện bất thường và chặn lại, đi kèm với tỷ lệ phần trăm gian lận thực tế trên tổng luồng giao dịch.



Hình 2.2: Giao diện hiển thị các thẻ chỉ số trọng yếu (KPIs) theo thời gian thực

Hệ thống Biểu đồ Trực quan hóa (Visualizations)

Các biểu đồ trên giao diện được tối ưu hóa bằng các thư viện đồ họa để xử lý luồng dữ liệu liên tục đổ về mà không gây hiện tượng nghẽn hay chậm giao diện:

- **Biểu đồ chuỗi thời gian (Time-series Area Charts):** Bao gồm Biểu đồ Tổng số giao dịch thể hiện đà tăng trưởng lũy tiến và Biểu đồ Khối lượng giao dịch (\$) phản ánh các biến động xung nhịp của dòng tiền thực tế theo từng giây. Các biểu đồ này hỗ trợ phát hiện các đợt tấn công quá tải giao dịch (burst traffic).



Hình 2.3: Biểu đồ chuỗi thời gian giám sát số lượng và khối lượng dòng giao dịch

- **Biểu đồ Phân phối (Distribution Charts):** Biểu đồ vành khuyên (Donut Chart) phản ánh trực quan tỷ lệ phân lớp mất cân bằng nghiêm trọng giữa giao dịch an toàn và gian lận. Bên cạnh đó, biểu đồ cột (Histogram) phân phối điểm số rủi ro minh họa rõ ràng tần suất xuất hiện của các giao dịch bất thường nằm ở dải điểm nguy hiểm cao từ 90 đến 100.



Hình 2.4: Biểu đồ phân phối trạng thái giao dịch và dải điểm số rủi ro hệ thống

Bảng giám sát luồng giao dịch và Trạng thái đường ống

Để hỗ trợ nghiệp vụ rà soát, kiểm toán và quản trị hạ tầng, phân hệ cuối cùng thực hiện bóc tách luồng dữ liệu thành hai bảng riêng biệt:

- **Recent Transactions:** Lọc hiển thị dòng giao dịch bình thường với xác suất gian lận gần như bằng 0%, trạng thái APPROVED.
- **Fraud Alerts:** Cô lập danh sách các giao dịch có điểm rủi ro chạm ngưỡng từ 90 đến 99, xác suất gian lận vượt mức 98% và lập tức dán nhãn nguy hiểm CRITICAL.
- **Giám sát sức khỏe Đường ống (Pipeline Status):** Tích hợp các thẻ trạng thái kiểm tra liên tục (Active) kết nối của Kafka Broker, Spark Engine và PostgreSQL, giúp kỹ sư vận hành nhanh chóng khoanh vùng nếu xảy ra sự cố kỹ thuật.

Recent Transactions

Transaction ID	Amount	Type	Fraud Probability	Risk Score	Status	Time
TXN_00023562	\$3.99	transfer	0.00%	0	APPROVED	13:54:01
TXN_00023829	\$150.00	withdraw	0.01%	0	APPROVED	13:54:01
TXN_00023699	\$68.90	online_purchase	0.02%	0	APPROVED	13:54:01
TXN_00023573	\$399.00	payment	0.00%	0	APPROVED	13:54:01
TXN_00023944	\$20.00	topup	0.00%	0	APPROVED	13:54:01
TXN_00023832	\$1.00	withdraw	0.03%	0	APPROVED	13:54:01
TXN_00023653	\$9.99	payment	0.00%	0	APPROVED	13:54:01
TXN_00023915	\$67.50	online_purchase	0.00%	0	APPROVED	13:54:01
TXN_00023535	\$439.00	topup	0.06%	0	APPROVED	13:54:01
TXN_00023511	\$115.30	payment	0.00%	0	APPROVED	13:54:01

Fraud Alerts

Transaction ID	Amount	Fraud Probability	Risk Score	Severity	Time
TXN_00022640	\$1.00	96.87%	96	CRITICAL	13:53:54
TXN_00020198	\$104.81	99.82%	99	CRITICAL	13:53:30
TXN_00018773	\$0.68	100.00%	99	CRITICAL	13:53:18
TXN_00018809	\$30.30	98.85%	98	CRITICAL	13:53:18
TXN_00018472	\$0.68	99.97%	99	CRITICAL	13:53:14
TXN_00018466	\$0.68	99.96%	99	CRITICAL	13:53:14
TXN_00017366	\$99.99	99.99%	99	CRITICAL	13:53:05
TXN_00017480	\$99.99	99.98%	99	CRITICAL	13:53:05
TXN_00017453	\$99.99	99.98%	99	CRITICAL	13:53:05
TXN_00017407	\$99.99	99.98%	99	CRITICAL	13:53:05

Hình 2.5: Bảng cô lập hồ sơ gian lận và hệ thống giám sát sức khỏe các dịch vụ lõi

CHƯƠNG 3: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

3.1. Kết luận

Dự án đã hoàn thành mục tiêu nghiên cứu và xây dựng thành công đường ống dữ liệu (Data Pipeline) toàn diện phục vụ bài toán phát hiện gian lận thẻ tín dụng theo thời gian thực trong môi trường Mobile Banking. Thông qua quá trình thực nghiệm, hệ thống đã chứng minh được những ưu điểm sau:

- **Hạ tầng dữ liệu lớn mạnh mẽ:** Sự kết hợp giữa Apache Kafka làm hệ thống hàng đợi thông điệp thông lượng cao và PySpark Structured Streaming để xử lý luồng theo mẻ nhỏ (Micro-batching) giúp hệ thống vận hành bền bỉ, xử lý mượt mà dòng chảy dữ liệu liên tục mà không gặp hiện tượng thất cổ chai bộ nhớ.
- **Mô hình học máy tối ưu:** Thuật toán XGBoost Classifier kết hợp với kỹ thuật tính toán trọng số lớp thiểu số ($scale_pos_weight \approx 577.29$) đã giải quyết triệt để thách thức mất cân bằng dữ liệu cực đoan ($< 0.2\%$). Mô hình đạt chỉ số ROC-AUC ấn tượng lên tới 0.9778, phát hiện chính xác 84% số vụ gian lận thực tế và giảm thiểu tối đa tỷ lệ báo động sai.
- **Giải pháp kỹ thuật thực tiễn:** Nhóm đã xử lý thành công các lỗi hệ thống nghiêm trọng như hiện tượng nghẽn luồng PyArrow bằng cách tái kiến trúc tính toán tập trung trên Spark Driver, đồng thời xử lý triệt để sai số dấu phẩy động bằng cơ chế ép kiểu làm tròn dữ liệu.
- **Giám sát trực quan tức thời:** Dashboard Streamlit tích hợp cơ chế Polling và các cầu phần đồ họa thông minh (Risk Gauge Chart) đã chuyển đổi thành công các chỉ số kỹ thuật khô khan thành công cụ quản trị rủi ro trực quan, hỗ trợ đắc lực cho nghiệp vụ an ninh ngân hàng.

3.2. Hướng phát triển

Để nâng cao hơn nữa hiệu năng và đưa hệ thống tiệm cận với môi trường vận hành thực tế tại các tổ chức tài chính, các hướng phát triển tiếp theo của dự án bao gồm:

- **Mở rộng hạ tầng và điều phối tự động:** Chuyển đổi kiến trúc Docker Compose cục bộ hiện tại sang nền tảng điều phối container chuyên nghiệp bằng Kubernetes (K8s). Giải pháp này cho phép hệ thống triển khai linh hoạt trên các môi trường điện toán đám mây như AWS hoặc GCP, tự động mở rộng quy mô (Auto-scaling) khi lưu lượng giao dịch trong các khung giờ cao điểm tăng vọt.
- **Nâng cấp thuật toán và làm giàu đặc trưng:** Thử nghiệm tích hợp các mô hình học sâu (Deep Learning) tiên tiến như mạng nơ-ron hồi quy LSTM để phân tích chuỗi hành vi mua sắm dài hạn của khách hàng, hoặc mạng nơ-ron đồ thị (GNN) nhằm bóc tách các đường dây tội phạm công nghệ cao có tổ chức. Đồng thời, làm giàu mô hình bằng các đặc trưng ngoại vi như Device Fingerprint (dấu vết thiết bị) và định vị GPS.
- **Xây dựng cơ chế phản hồi và tự động khóa thẻ (Auto-block):** Nâng cấp hệ thống từ mức độ cảnh báo (Alerting) sang mức độ chủ động ngăn chặn (Prevention). Thiết lập một đường luồng API phản hồi ngược (Webhook) kết nối trực tiếp vào lõi Core Banking để thực hiện khóa thẻ tạm thời ngay lập tức khi điểm số rủi ro giao dịch vượt ngưỡng an toàn.

TÀI LIỆU THAM KHẢO

1. Machine Learning Group - ULB (2013). *Credit Card Fraud Detection Dataset*. Kaggle.
2. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794).
3. Apache Software Foundation. *Apache Kafka Documentation - Distributed Message Queue*.
4. Apache Software Foundation. *PySpark Structured Streaming Programming Guide*.
5. Streamlit Inc. *Streamlit Documentation - Open-source app framework for Data Science*.

PHÂN CÔNG NHIỆM VỤ

Bảng 3.1: Bảng phân công công việc các thành viên trong nhóm

Họ và tên	Mã sinh viên	Công việc đảm nhận
Doãn Nam Khánh	24022360	Thiết lập Ingestion Layer (Kafka Producer), tiền xử lý dữ liệu từ file CSV. Thiết kế cơ sở dữ liệu PostgreSQL, Cấu hình luồng Stream (Spark), huấn luyện Machine Learning,.
Đỗ Đức Hùng	24022342	Lập trình giao diện Dashboard giám sát, viết báo cáo kỹ thuật. Tối ưu hóa thuật toán, Docker hóa hệ thống, thực hiện kiểm thử và làm Slide